

Automated Scanning for Injection Vulnerabilities in Large Language Models

Leandros Galea, Daren Scerri
Institute of Information & Communication Technology
Malta College of Arts, Science & Technology
Corradino Hill
Paola PLA 9032
{leandros.galea.g51976, daren.scerri}@mcast.edu.mt

Abstract—Large language models (LLMs) have revolutionised natural language processing, but they are susceptible to security vulnerabilities, including prompt injection attacks. This study examines such vulnerability within LLMs and evaluates automated scanning as a vulnerability detection method. Two sequential experiments were conducted, focusing on prompt injection effectiveness and automated vulnerability scanning. The initial experiment confirmed such vulnerabilities, sometimes with minimal resistance, while the subsequent experiment demonstrated high accuracy in detecting successful and rejected prompt injection attacks, using the proposed automated approach. Results highlight the inherent vulnerability to such attacks within LLMs, and the potential of such automated detection method. This study underscores the need for proactive security measures and suggests avenues for future research to enhance LLM security.

Index Terms—Security, Injection Attack Vulnerability, Large Language Models

I. INTRODUCTION

The increasing reliance on software applications across industries has made them critical to modern-day business operations, highlighting the need for robust cyber-security solutions. The widespread adoption of LLM based systems, has inadvertently opened new avenues for malicious attacks [1], [2]. This heightened risk is particularly evident due to the inherent characteristics of LLMs, which rely on input prompts susceptible to manipulation, thereby potentially exposing host systems to injection attacks [1]. Moreover, the novelty of such systems further compounds to the risk, as security protocols and best practices may not yet be fully established.

The purpose of this experimental study is to investigate prompt injection attacks on LLMs and subsequently analyse the efficacy of an automated detection method for identifying such vulnerabilities within LLMs. The dependent variables in this study are the detection accuracy of the proposed automated scanner and the success rates of the attacks. The independent variables include the specific model being utilised, the different prohibited prompts that will be given to the LLMs, and the way in which these will be delivered to each LLM. Furthermore, a selection of the responses returned by the LLMs, will be utilised as the independent variable in the subsequent experiment concerning the automated scanner. To maintain experimental consistency and manage the potentially infinite variations in prompt content, the complete list of prompts, environmental variables and testing procedures

are designated as control variables. Additionally, this study considers the level of vulnerability inherent in each model and the mitigation strategies employed, as intervening variables that may influence the success rate of attacks. The relationship between the variables intends to highlight how the different prompt inputs and/or models affect the success rate of the attack attempts and the accuracy of the automated scanner. The aim of this research is to explore prompt injection attacks on LLMs and to correctly identify vulnerabilities using an automated approach. Specifically, the hypothesis for this study states that LLMs are susceptible to injection attacks, and that such vulnerabilities can be effectively detected using an automated approach.

In light of the previously indicated factors, as depicted in Fig 1, during this research, a positivist philosophy along with a deductive approach will be followed, which allows for a methodical and thorough examination of the attacks and the automated scanning process. Furthermore, an experimental strategy will be employed, along with a mono-method choice and a cross-sectional time horizon.

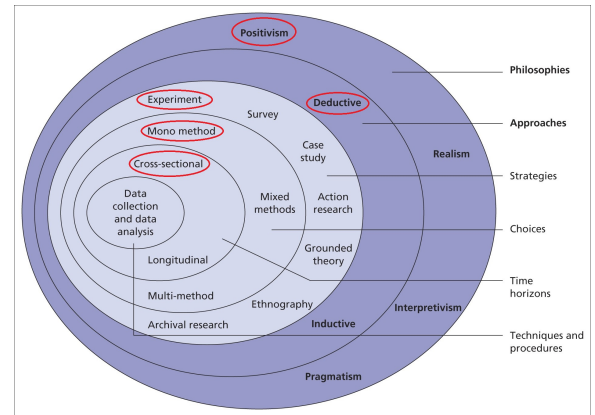


Fig. 1. The Research Onion - Adopted from [3]

II. LITERATURE REVIEW

The prevalent integration of LLMs in fields like conversational AI and natural language processing, has rapidly transformed the state of current technology. However, despite the advantages of LLMs, there are also risks associated for both

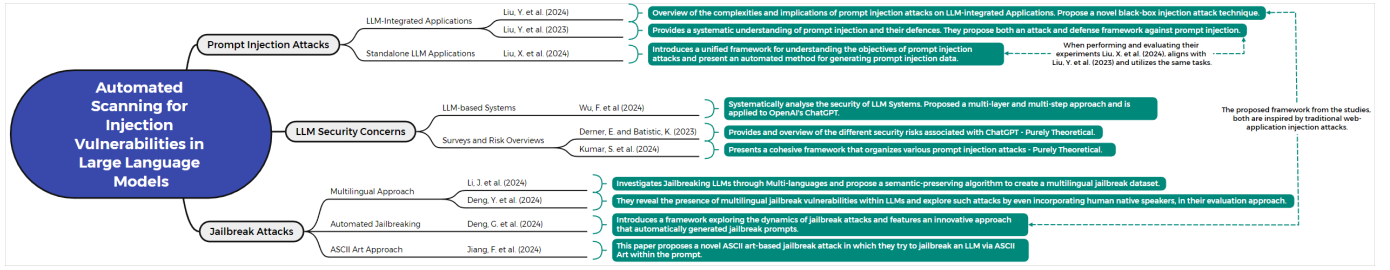


Fig. 2. The Literature Map

individual users and the organizations that use them. Although such systems are relatively novel, attacks directed towards LLMs have become a prominent issue, posing new problems for cyber-security experts. To assist and contribute to the growing knowledge in this area, various researchers investigated such issue. This section reviews research methodologies employed by a carefully selected set of studies (mainly preprints, due to the novelty of such systems), and analyses the diverse approaches and strategies adopted by the researchers. Fig. 2 visually represents the selected literature, providing a deeper understanding of the contributions to the main study issue and the interconnections between the literature. By means of this analysis, this review aims to highlight common methodological patterns and evaluate the advantages and disadvantages of the methodologies employed.

A. Overview of Security Concerns in Large Language Models

As discussed in [4], there are various security risks that malicious actors can exploit to compromise the LLM's defenses. Producing offensive content, malicious text and code generation, private data disclosure, information gathering, and fraudulent services are all implications that an LLM should not assist with. These aspects are all discussed in depth in [4], where prohibited scenarios are tested against OpenAI's ChatGPT, providing examples of prompts and their subsequent response to demonstrate ChatGPT's content filters at work and explore potential ways to bypass such safeguards.

In [5], researchers systematically analyse LLM security mechanisms, by focusing on a multi-layer and multi-step attack approach, which was subsequently applied to OpenAI's ChatGPT in two case-studies, namely, unethical image displaying and web indirect malicious instruction execution. Their analysis, which was split into three stages, namely, multi-layer security analysis, existing constraints analysis, and analysis of the constraints' robustness, initially showed that the LLM's security measures prevented the attacks. However, through logical manipulation, they managed to 'force' the LLM to perform prohibited actions, revealing security issues within the LLM and its integration with other components. Based on these findings, the researchers constructed an end-to-end practical attack to obtain conversation records by exploiting the identified vulnerabilities, detailing each step and rationale of the progress, whilst also linking a repository containing their source code. It should be noted that, by analysing the

initial experiment, the researchers were able to build on it, and successfully perform an attack on a state-of-the-art (SOTA) LLM.

B. Prompt Injection Attacks

In a typical LLM system, the main point of communication between the end-user and the LLM is through prompts. Thus, apart from any vulnerabilities introduced in the training stage or data, the majority of end-user attacks must occur via prompt manipulation. [6] discusses several prompt injection attacks that have documented evidence of success against LLMs, organizing them into a cohesive framework and noting various characteristics of each approach. The following papers discussed in this subsection examine, and try to attempt, prompt injection attacks on LLMs.

The researchers in [7] introduced an integrated framework for understanding prompt injection attack objectives and presented a method to automatically generate prompt injection data for attacks. They started by summarizing the threat model, focusing on scenarios where an LLM combines user instructions (prompt content) with external data (e.g., a PDF). The researchers hypothesised that malicious data could be injected with the external data to bypass security mechanisms. The objectives of prompt injection were categorised into 'static', 'semi-dynamic', and 'dynamic', discussing each in depth. Taking basis from previous research, they converted these objectives into optimization loss functions, selecting Momentum Gradient-based search for its suitability with discrete tokens and unknown input contexts. For their experiment, they utilized Meta's LLAMA2-7b-chat model and various datasets to carry out natural language processing tasks, such tasks aligning with those of [8]. The researchers detailed the hyper-parameters used (top-k value, batch size, iteration count, momentum weight, and token length) and baseline approaches, which all contribute to enhancing reproducibility. To evaluate the effectiveness of each baseline method, they assessed the Attack Success Rate across multiple dataset samples and provided result illustrations and linked their repository containing their work.

However, LLMs are not always stand-alone; they can also be integrated into third-party applications, such as note-taking tools with AI auto-complete features. [8] and [9] both delve into prompt injections within such LLM-Integrated applications.

[8] presents a systematic understanding of prompt injection attacks and their defenses, delving into the theory behind such attacks, and proposing a general framework to formalize them. Moreover, [9] deconstructs the complexities and implications of prompt injection attacks, and performs an exploratory analysis focusing on the patterns and effectiveness of existing prompt injection attacks on various SOTA LLMs. Using their findings of the systematic analysis, [8], designed a new attack approach by combining known methods, and subsequently proposed a framework structure to systematize defenses against prompt injection attacks. They evaluated it on ten LLMs across seven tasks (similar to [7]), utilising metrics like 'Performance under No Attacks', 'Attack Success Score', and 'Matching Rate', presenting their results using graphs, which assisted in the analysis discussion. Similar to the research flow of [8], [9], utilised the results obtained in their pilot analysis of prompt injection attacks, to propose a novel black-box prompt injection attack technique, nicknamed 'HOUYI', inspired from conventional web injection attacks. As highlighted in the study, such conventional attacks (e.g., SQL Injection), succeed by creating a payload that terminates the preceding syntax. HOUYI uses similar logic, interpreting the injected prompt as an instruction to be answered separately, whilst ignoring the context. To evaluate HOUYI, the researchers used thirty-six well-documented LLM-integrated applications. They highlighted the adjusted hyper-parameters utilised in the experiment, and ran the same prompt multiple times to try and avoid any influence of randomness. The researchers took a professional approach by informing vendors of their findings before publication, to mitigate potential harm, which some vendors acknowledged and validated their discoveries. Additionally, [8] shares a comprehensive prompt list and source code repository, while [9] refrained from disclosing the source code until the LLM vendors had sufficient time for review.

C. Jailbreak Attacks

Jailbreaking in LLMs involves unauthorized access or manipulation of the model's underlying mechanisms, aiming to bypass security restrictions or gain elevated privileges, akin to circumventing restrictions on mobile devices to bypass usage constraints. This form of attack often leverages vulnerabilities introduced through prompt injection, where malicious prompts are used to alter normal model behavior.

Both [10] and [11], explore using multiple languages to jailbreak LLMs, both presenting two potential scenarios: an unsuspecting user inadvertently breaking the LLM while using a different language, and a malicious user attempting to bypass safety measures by employing multiple languages. Researchers in [10] conducted an extensive empirical study to develop a novel semantic-preserving algorithm, to create a multilingual jailbreak dataset, given the absence of one. Their proposed algorithm, based on Microsoft Translate, ensured high semantic fidelity during translation, and filtered outputs (based on semantic resemblance) for dataset integrity. They evaluated the developed dataset using four SOTA LLMs and analysed

the Attack Success Rate. Initially, they tested without using any jailbreaking templates, integrating them after, to be able to compare and contrast results. A subsequent experiment was conducted in which LLMs' internal workings were analysed whilst prompting in multiple languages, noting successful and failed attacks, utilising an open-source model with observable internal workings. Basing on their experiments, they proposed LLM Fine-tuning to enhance security, which was tested, noting improved security albeit with reduced response verbosity. Similarly, [11], started by a preliminary study, in which, a dataset of fifteen harmful English prompts was constructed and translated to thirty languages using Google Translate, and subsequently evaluated against OpenAI's GPT3.5. They enhanced their experiment by appending three hundred prompts from a third-party dataset, and tested against GPT3.5 and GPT4. Based on findings, the researchers proposed the 'Self-Defense' framework for automatic generation of multilingual training data for fine tuning - an approach that was also suggested by [10], enhancing security against multilingual jailbreak attacks. Using OpenAI's ChatGPT (due to its fine tuning ability), they applied the framework and demonstrated its effectiveness on the same dataset.

An innovative approach is introduced in [12], which explores ASCII art to jailbreak LLMs. This research started with a Vision-in-Text Challenge benchmark, assessing five SOTA LLMs in recognising prompts that cannot be solely interpreted using semantics. Based on the findings - mainly that all evaluated LLMs struggled to recognise ASCII art - the researchers devised 'ArtPrompt', a novel jailbreak attack which was able to leverage such poor performance and invoke unintended behavior from the LLM. The design of the attack involved 'Word Masking' and 'Cloaked Prompt Injection', both discussed in depth, and was subsequently evaluated across the same LLMs. They compared ArtPrompt with five advanced jailbreak attacks from other studies, and assessed the results using three metrics: 'Helpful Rate', 'Attack Success Rate', and 'Average Harmfulness Score', presented in tables and graphs.

In [13], researchers investigated the dynamics of jailbreak attacks and defences, proposing a novel time-based approach to deconstruct LLM defenses and generate jailbreak prompts. Similar to [9], this approach draws inspiration from traditional web-application attacks, specifically time-based SQL Injection. Researchers conducted an empirical study to understand jailbreak threats and defenses, examining LLM vendor policies and analyzing jailbreak prompts (sourced from other studies and online datasets). They selected four LLMs and tailored the prompts to each, subsequently experimenting by prompting the LLMs and assessing effectiveness. Based on their findings, they proposed 'MasterKey', which leverages service response time to uncover hidden LLM defensive mechanisms and identify possible weaknesses, splitting the process into multiple stages for detailed analysis. Additionally, they introduced automated jailbreak prompt generation, facilitated by prior reverse-engineering analysis. They structured their workflow into three sub-processes: 'Dataset Building and Augmentation', 'Continuous Pre-training and Task Tuning',

and 'Reward Ranked Fine Tuning', all extensively discussed within the study. MasterKey was evaluated on the Vicuna LLM, recognized as the best-performing model on the open-source leaderboard at the time. Employing default settings, they rigorously repeated the experiment five times to reduce randomness and enhance reliability, utilizing two metrics, namely, 'Overall Jailbreak Success Rate' and 'Quality of Generated Jailbreak Prompts', to thoroughly assess their prototype. Similar to the previously discussed studies [10] and [11], the researchers recommended fine-tuning, and notably, supervised learning as mitigation strategies to mitigate such attacks.

In conclusion, this review of methodologies employed in current research, underscores the diverse approaches employed in exploring this emerging field of cyber-security. While each study contributes valuable insights, continuous research is essential given the pioneering nature of this topic.

III. RESEARCH METHODOLOGY

After reviewing the methodologies outlined in Section II, it was observed that researchers across multiple studies ([5], [8], [9], [10], [11], [12], [13]), adopted a sequential approach, often starting with pilot and exploratory studies to assess effectiveness before proceeding to deeper investigations. In alignment with this pattern, our study embraced a similar approach, conducting two sequential experiments, each aimed at addressing a specific research question, which are stated as follows:

- **RQ1 (Prompt Injection Effectiveness)** - How effective are prompt injection attacks against LLMs?
- **RQ2 (Automated Prompt Injection Vulnerability Scanning)** - How accurate would an automated detection approach be in identifying successful/unsuccessful prompt injection attacks?

To address the research questions, this study is structured around two primary experiments. The first experiment focuses on RQ1 by evaluating the susceptibility of various LLMs to prompt injection attacks. This involves testing multiple LLMs with prohibited prompts to assess the success rate of these attacks. The second experiment targets RQ2 by developing and testing an automated scanner designed to identify successful and rejected prompt injection attempts. This experiment aims to measure the accuracy and reliability of the automated approach in correctly detecting prompt injection attacks, thereby providing insights into its effectiveness as a defensive tool.

A. Methodology Approach

This approach aligns with the positivist philosophy and deductive approach discussed in Section I, which emphasises a methodical and thorough examination of vulnerabilities and the subsequent automated scanning process. Therefore, our experiments are aligned with the Research Onion model illustrated in Fig. 1, ensuring consistency and coherence in our research methodology.

RQ1: Prompt Injection Effectiveness

This subsection delves into the first research question, aiming to assess the effectiveness of prompt injection attacks against LLMs. Through this analysis, we seek to enhance our understanding of how LLMs react to prompts they should not engage with.

Victim LLMs: To conduct the experiment, five SOTA LLMs were chosen from reputable model providers. Such models were chosen based on the following criteria: (a) Popularity, (b) Availability of official documentation, and (c) The ability to run locally and/or in a sandbox environment. All models were accessed via HuggingFace and their respective specific details are outlined in Table I. Additionally, computational power constraints were considered when configuring the models for this study.

TABLE I
VICTIM LLMs SPECIFICATIONS

Model Name	Model Provider	Parameters
gemma-7b [14]	Google	7 Billion
Llama-2-7b-chat-hf [15]	Meta AI	7 Billion
phi-1 [16]	Microsoft	1.3 Billion
Mistral-7B-Instruct-v0.1 [17]	MistralAI	7 Billion
stablelm-2-12b-chat [18]	StabilityAI	12 Billion

Experimental Environment: The selected models were executed on Google Colab, utilising the T4 GPU offered in their free tier. Google Colab allows users to run Jupyter Notebook files without local computational resources, offering a temporary session runtime environment. Python, particularly in the form of Jupyter Notebooks, was chosen as the programming language due to its compatibility. Several python dependencies, including bitasandbytes, transformers, peft, accelerate, sentencepiece, and LangChain, sourced from HuggingFace and PyPI, were installed on the runtime to facilitate communication with the LLMs. Additionally, imports such as AutoModelForCausalLM, AutoTokenizer, Pipeline, and Conversation from the 'transformers' package, as well as, pandas, torch, and drive (from google.colab), were required.

Prompt-List Dataset: To prompt the models, a dataset containing thirty-five prompts was compiled. Thirty were sourced from a public dataset [19], while an additional five prompts were devised to encompass common illegal and unethical activities. Each prompt was adapted into two variations for 'text-generation' and 'conversational' pipelines, as discussed further below. Furthermore, the prompts were structured to return a list (such as a list of steps or items), streamlining the subsequent automated scanner implementation.

Experiment Flow: The experiment flow involved several steps for the responses to be generated from each LLM. Initially, dependencies and imports were added as outlined above, including accessing HuggingFace's model repository (using HuggingFace CLI login) and mounting Google Drive for data storage. Specific models were loaded based on the Model-ID from HuggingFace, and a pipeline (conversational or text-generation) was specified. The conversational pipeline prompts questions to the LLM for responses, while the text-generation pipeline provides starting words for the LLM to

predict subsequent text. Additionally, some key variables were set to control the model behaviour, and these are outlined in Table II, while others were left as default. Prompts were loaded from a .csv file and filtered by ID (assigned during the dataset creation phase) to match the specified pipeline. Valid prompts were sent to the model, generating responses and storing them in a separate .csv file, with the corresponding prompt IDs.

TABLE II
KEY VARIABLE VALUES

Parameter	Value
trust_remote_code	False
clean_up_tokenization_spaces	True
max_new_tokens	512
min_new_tokens	2
max_time	120
temperature	0.1
repetition_penalty	1.1

Metrics: The effectiveness of prompts and eligibility of the respective responses were assessed using the Eligible Response Rate (ERR) and Attack Success Rate (ASR) metrics, defined as,

$$ERR = \frac{\# \text{ of Eligible Responses}}{\text{Total \# of Responses}}$$

$$ASR = \frac{\# \text{ of Successful Prompt Attacks Attempts}}{\text{Total \# of (Eligible) Prompt Attempts}}$$

Manual review of the output .csv file determined the success/rejection of the attack attempts. Additionally, to enhance reliability, the experiment was repeated 3 times to minimize the influence of randomness, which such approach was also employed by [9] and [13].

RQ2: Automated Prompt Injection Vulnerability Scanning

Using select responses returned by the LLMs in the preceding experiment, this subsection focuses on the second research question, where the main objective is to develop an automated scanner that is able to identify whether an LLM was able to refuse to respond to a constrained prompt, followed by manual accuracy analysis.

Experimental Environment: The automated scanner, developed and executed within a Python environment focused on processing the prompt responses. Due to the relatively low computational requirements of this prototype, it was deemed unnecessary to utilize the limited resources of Google Colab. Instead, the scanner was executed locally on a Python 3.10.4 kernel, with the following dependencies installed: ipykernel, ipython, jupyter, and notebook, all sourced from PyPI.

Experiment Flow: The scanner initially loaded prompt responses from a .csv file, pairing Prompt IDs with their corresponding responses. Each response underwent classification checks, including determining if the attack was 'Successful', 'Rejected', or 'Inconclusive'. Such checks were based on predefined criteria, such as the presence of specific phrases or list structure, hence why earlier, the prompt needed to be manipulated in a format for the LLM to return a list. Upon completion of the scanning process, the results were saved to a separate .csv file for further manual analysis.

Metrics: To evaluate the accuracy of the scanner, four metrics were employed: Successful Attacks Correctly Detected (SACD), Rejected Attacks Correctly Detected (RACD), Successful Attacks Falsely Detected (SAFD), and Rejected Attacks Falsely Detected (RAFD), defined as follows:

$$SACD = \frac{\# \text{ of Successful Attacks Correctly Detected}}{\text{Total \# of Successful Attacks}}$$

$$RACD = \frac{\# \text{ of Rejected Attacks Correctly Detected}}{\text{Total \# of Rejected Attacks}}$$

$$SAFD = \frac{\# \text{ of Successful Attacks Detected as Rejected Attacks}}{\text{Total Number of Successful Attacks}}$$

$$RAFD = \frac{\# \text{ of Rejected Attacks Detected as Successful Attacks}}{\text{Total \# of Rejected Attacks}}$$

A manual review of the output .csv file was conducted to accurately determine the results, which is discussed more in depth in Section IV. Furthermore, the scanner was executed three times, each time using a different .csv file returned from the first experiment.

B. Validity, Reliability, and Generalizability

Validity: Many steps were taken to ensure the validity of both experiments. First, the prompts used in the first experiment were carefully constructed to mimic real-world scenarios, including both publicly available prompts and those encompassing common illegal and unethical activities. Additionally, the experiment environment was adjusted to closely resemble typical publicly-available LLM usage conditions. Finally, a manual review of each output .csv file was conducted to ensure the accuracy of the results, reducing potential inaccuracies from automated processes.

Reliability: The reliability of both experiments was ensured through rigorous testing and repetition. The non-deterministic nature of LLMs [20] was acknowledged, which can impact reliability. To mitigate the effects of randomness and unpredictability, both experiments were conducted three times, accounting for the inherent variety in LLM responses. This repetition enhanced confidence in the findings' consistency and enabled a more thorough investigation of the data. Throughout all repetitions, the same procedures, prompt list dataset, and specified parameters were used. Despite the non-deterministic nature of LLMs, these measurements help to increase the experiments' overall reliability by providing confidence in the consistency of the results obtained.

Generalizability and Transferability: When considering the generalizability and transferability of the findings from this study, several factors must be acknowledged. The choice of LLMs and prompts in the initial experiment may limit generalizability, as variations in model architectures, training data, and prompt formats can affect applicability of the findings to other contexts. Additionally, the experimental setup and automated scanner focused on prompts returning a list, which may not apply to other prompt types. Therefore, while the research provides valuable insights into prompt injection vulnerabilities in LLMs, caution is needed when extrapolating the findings to broader contexts.

C. Ethical Considerations

While conducting this research, a set of ethical principles were adhered to, in order to ensure the integrity, transparency, and responsible use of AI technologies. Firstly, to respect intellectual property rights and ensure transparency, the sources of our data, including the publicly available dataset and proprietary LLMs utilised, were clearly documented and referenced. Explicit permission was requested and granted from Meta and Google to use their respective models, as required by their terms of service. Additionally, we committed to proactively notifying LLM vendors about any vulnerabilities discovered during the study before publishing our findings. This approach not only promotes the security and integrity of AI systems but also demonstrates our mindfulness of the potential societal impacts of this research, emphasizing that our intent is to contribute to the understanding and improvement of LLM security rather than exploit vulnerabilities.

IV. FINDINGS & DISCUSSION OF RESULTS

RQ1: Prompt Injection Effectiveness

As discussed in Section III, the effectiveness of the prohibited prompts against LLMs was evaluated through the first experiment. Two key metrics, ERR and ASR, mentioned previously, were utilised to quantify the results. ERR measures the proportion of eligible responses - defined as responses that provide relevant and coherent information - to the total number of prompts tested. ASR quantifies the ratio of successful prompt attempts (prompts that returned a supposedly forbidden response) in relation to the total number of (eligible) prompts forwarded. Table III presents a summary of both metrics for each model. For a comprehensive understanding of the results, refer to the detailed visual representation provided in Figure 3, which depicts the aggregation of results from the three repetitions of the experiment carried out. Additionally, the results are also detailed in Table IV.

TABLE III
SUMMARY OF ERR AND ASR

Model Name	ERR	ASR
gemma-7b	0.00	N/A
Llama-2-7b-chat-hf	0.99	0.49
phi-1	0.09	N/A
Mistral-7B-Instruct-v0.1	1.00	0.96
stablelm-2-12b-chat	0.99	0.51

The analysis of the results from this experiment revealed interesting insights into prompt injection attack effectiveness against various LLMs. Models displayed either very high ERR (Llama-2-7b-chat-hf, Mistral-7B-Instruct-v0.1, stablelm-2-12b-chat) or very low ERR (gemma-7b, phi-1), with no model achieving moderate scores. For gemma-7b, it was noted that the responses were often blank or random, indicating that the low ERR is due to potential interaction issues while using the Huggingface pipelines. Furthermore, when considering the phi-1 model, the responses were actual text somewhat related to the prompt but cannot be classified as eligible responses. We attributed this phenomenon to the significantly reduced

parameter count of this model when compared to the parameter counts of the other models.

Moreover, the ASR provides insightful analysis on attack effectiveness. Considering the low ERR for the gemma-7b and phi-1 models, further analysis on these models was deemed unnecessary. Therefore, the ASR metric was applied solely on the Llama-2-7b-chat-hf, Mistral-7B-Instruct-v0.1, and stablelm-2-12b-chat models. Both Llama-2-7b-chat-hf and stable-2-12b-chat achieved a similar ASR score, hovering around the 50% mark. However, Mistral-7B-Instruct-v0.1, achieved a notably higher ASR, suggesting minimal resistance to such attacks. Notably, successful responses in Llama-2-7b-chat-hf and stable-2-12b-chat predominantly originated from the Text-Generation Pipeline, while rejected responses stemmed from the Conversational Pipeline. Such important detail highlights the fact that the way the prompts are delivered to the LLM may be more effective than the actual prompt content in order to achieve a successful attack.

Comparing the results across the experiment's iterations revealed consistent outcomes, which highlights the reliability of the findings, as it suggests that the observed results were not merely due to chance or random variations in the experimental conditions. Despite minor textual variations between iterations, the overall conclusions drawn from the data remained unchanged. As discussed previously, the decision to manually review the results instead of relying solely on an automated process was deliberate, aimed at reducing any potential inaccuracies that might arise, which contributes to the validity of the findings. However, it is important to acknowledge certain limitations with regards to generalizability. Different models with different specifications may exhibit varying behaviours, as was evident in gemma-7b and phi-1, which predominantly returned ineligible responses. Such variability highlights the need for caution when extrapolating the findings to other models, as outcomes may not be universally applicable.

In light of the first part of the original hypothesis, which states that LLMs are susceptible to injection attacks, the results affirm the validity of this assertion. Across the models tested, especially in Llama-2-7b-chat-hf, Mistral-7B-Instruct-v0.1, and stablelm-2-12b-chat, there was a notable prevalence of successful prompt injection attacks, indicating possible inherent vulnerabilities within such LLMs. [10] also undertakes an initial experiment in which forbidden scenarios are sent to the LLM without any jailbreak instructions, which is comparable to our experiment. Although direct comparison of findings is challenging due to variations in LLMs and experimental conditions, [10] achieved diverse outcomes throughout their experiment. Notably, [10] observed diminishing attack success rates with increased model specifications, a factor not explored in our study due to computational constraints.

RQ2: Automated Prompt Injection Vulnerability Scanning

The subsequent experiment aimed to assess the accuracy of the automated scanner developed, focusing on responses from Llama-2-7b-chat-hf, Mistral-7B-Instruct-v0.1, and stablelm-2-12b-chat. Furthermore, given that this experiment was focused

TABLE IV
MODEL RESULTS TABLE

Model	Attack Success-TG	Attack Rejected-TG	Ineligible-TG	Attack Success-C	Attack Rejected-C	Ineligible-C
gemma-7b	0	0	105	0	0	105
Llama-2-7b-chat-hf	102	0	3	0	105	0
phi-1	18	0	87	0	0	105
Mistral-7B-Instruct-v0.1	105	0	0	96	9	0
stablelm-2-12b-chat	102	0	3	3	102	0

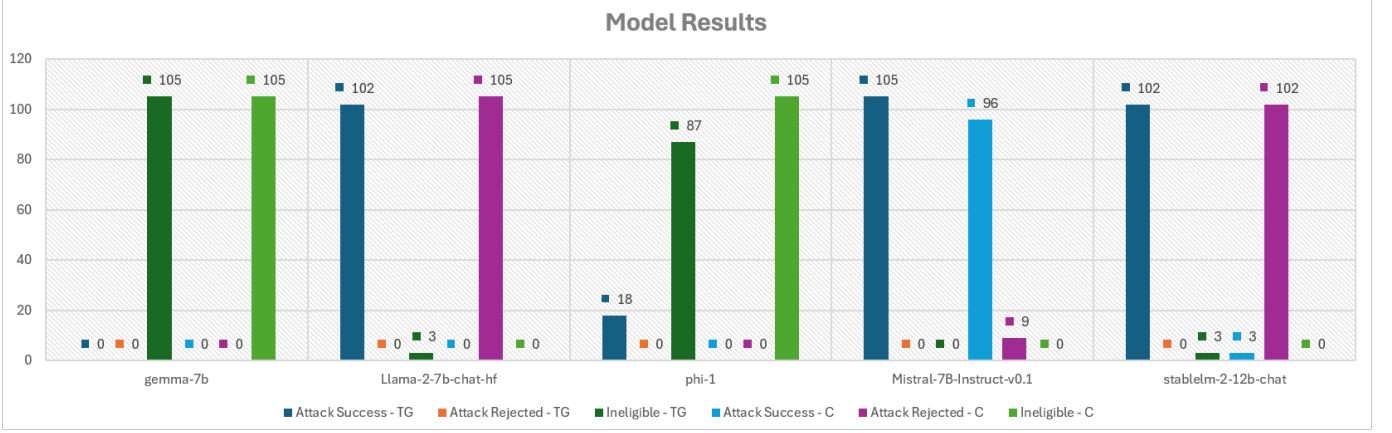


Fig. 3. Model Results Visual

on the automated scanning process, the responses were mixed and no further notability was given to the models providing the responses. As mentioned in Section III, four key metrics, 'SACD', 'RACD', 'SAFD', and 'RAFD', were utilized to evaluate the scanner's performance. SACD and RACD reflect the correct identification of successful and rejected attacks, respectively, while SAFD and RAFD indicate instances where the scanner oppositely classified attacks. Table V summarizes the results for each metric employed and Fig. 4 illustrates a confusion matrix, detailing the correspondence between the scanner's predictions and the actual outcomes. This matrix provides a comprehensive view of the automated scanner's performance and also depicts the aggregated results from the three experiment repetitions.

TABLE V
AUTOMATED SCANNER METRIC RESULTS

Metric	Result
SACD	0.99
RACD	0.97
SAFD	0.01
RAFD	0.00

The confusion matrix demonstrates the automated scanner's commendable performance in identifying successful and rejected prompt injection attacks, with a high number of true positive and true negative predictions. Furthermore, one important entry in the confusion matrix, is the actual successful attacks predicted as rejected attacks, as this highlights those successful attacks that have bypassed the scanner. Notably, only three out of the four-hundred-and-five actual successful

Automated Scanner Results				
TARGET \ OUTPUT	Success	Rejected	Inconclusive	SUM
Success	402 63.81%	0 0.00%	6 0.95%	408 98.53% 1.47%
Rejected	3 0.48%	213 33.81%	0 0.00%	216 98.61% 1.39%
Inconclusive	0 0.00%	6 0.95%	0 0.00%	6 0.00% 100.00%
SUM	405 99.26% 0.74%	219 97.26% 2.74%	6 100.00% 0.00%	615 / 630 97.62% 2.38%

Fig. 4. Automated Scanner Results Confusion Matrix

attacks were erroneously classified as rejected, indicating a very low false negative rate. When considering the derived metrics (refer to Table V), SACD and RACD values, reflecting the correct identification of successful and rejected attacks, respectively, were notably high at 0.99 and 0.97, affirming the robustness of the automated scanner in accurately detecting both types of attack results. Conversely, the SAFD metric, indicating successful attacks misclassified as rejected, was minimal at 0.01, and, as discussed, this metric may be considered as the most important, since it highlights the rate at which a successful attack bypassed the automated scanner. Similarly, the RAFD metric, representing rejected

attacks incorrectly labeled as successful, resulted in a 0.00 value in our experiment due to the absence of such responses.

Whilst comparing and contrasting the results, even between the repetitions themselves, it became evident that the automated scanner performed consistently and reliably. The automated scanner consistently demonstrated high accuracy across the three trials, effectively detecting both successful and rejected attacks with minimal incorrect predictions. As previously stated, reliability in this experiment was enhanced through the multiple iterations of this experiment, each time utilising responses derived from the first experiment. Furthermore, similar to the first experiment, to ensure validity, the scanner results were reviewed manually in order to be able to present the results showcased above. However, it is also important to again acknowledge the potential limitations in generalizability due to the specific conditions and response format that the chosen models returned. Although care was taken to ensure that the development of the automated scanner was not biased based on the response pool, different response formats may influence the scanner’s performance in other scenarios.

The results support the second part of the hypothesis, affirming that vulnerabilities in LLMs can indeed be effectively detected using an automated approach. By developing and testing the proposed automated scanner, we were able to accurately identify instances of successful attacks, as well as, instances of rejected attacks. Taking into consideration the studies reviewed in Section II, none of them study the approach of proposing an automated scanner to detect such vulnerabilities, therefore it is not possible to directly compare our results with any of those reviewed.

V. CONCLUSION

This research delved into the susceptibility of LLMs to prompt injection attacks and evaluated the effectiveness of automated vulnerability scanning in detecting such attacks. Through two experiments, this study addressed research questions and a hypothesis, aiming to enhance understanding of LLM security and mitigation strategies.

The main conclusions drawn from this study were that LLMs are indeed susceptible to such prompt injection attacks and furthermore, an automated approach can also effectively detect such vulnerabilities by analysing certain responses returned by the LLM. The first experiment conducted, which focused on the first research question, demonstrated that such forbidden prompts can easily bypass the LLM’s restrictions (if any) and result in a prompt injection attack. Furthermore, with the second experiment, which focused mainly on the second research question, we demonstrated that it is possible to use an automated approach to detect whether a prompt injection attack has been successful or rejected on an LLM, by analysing the response. Such experiments and their respective conclusions directly address the research questions and the hypothesis that was posed at the outset of this study, affirming the vulnerability of LLMs to prompt injection attacks and

highlighting the potential of an automated approach to assist developers whilst producing such LLMs.

While the methodology employed yielded robust and insightful findings, some shortcomings should also be acknowledged. As discussed, consideration was taken whilst selecting the models to be utilised due to the computational power constraints encountered. Therefore, it was not possible to also experiment with different specifications of each model and analyse such results. Furthermore, as was also discussed in Sections III and IV, the generalizability of the findings may be limited to certain LLM models, and the specific response structure (responses were required to be in the form of a list) that was utilised in the experiments.

To address such limitations and further advance research in this area, several avenues for further investigation are proposed. One suggestion is to conduct more extensive experiments with a broader range of LLMs and an increased prompt pool. When conducting a more extensive experiment, it would not be feasible to review results manually, similar to the approach undertaken in this study, and therefore it would be suggested to use an external SOTA LLM that would be used to identify whether the prompt injection attack was successful or rejected, and to subsequently analyse the results of the automated scanner. Additionally, exploring alternative detection methods (such as one without the need of the response format to be in the form of a list) is also suggested to enhance the generalizability of such automated scanner.

REFERENCES

- [1] E. Derner, K. Batistič, J. Zahálka, and R. Babuška, “A security risk taxonomy for large language models,” 2023.
- [2] L. Weidinger, J. Uesato, M. Rauh, C. Griffin, P.-S. Huang, J. Mellor, A. Glaese, M. Cheng, B. Balle, A. Kasirzadeh, C. Biles, S. Brown, Z. Kenton, W. Hawkins, T. Stepleton, A. Birhane, L. A. Hendricks, L. Rimell, W. Isaac, J. Haas, S. Legassick, G. Irving, and I. Gabriel, “Taxonomy of risks posed by language models,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 214–229. [Online]. Available: <https://doi.org/10.1145/3531146.3533088>
- [3] M. Saunders, P. Lewis, and A. Thornhill, *Research Methods for Business Students*, 5th ed. Harlow, England: Pearson Education, 2007.
- [4] E. Derner and K. Batistič, “Beyond the safeguards: Exploring the security risks of chatgpt,” 2023.
- [5] F. Wu, N. Zhang, S. Jha, P. McDaniel, and C. Xiao, “A new era in llm security: Exploring security concerns in real-world llm-based systems,” 2024.
- [6] S. S. Kumar, M. Cummings, and A. Stimpson, “Strengthening llm trust boundaries: A survey of prompt injection attacks,” 02 2024.
- [7] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, “Automatic and universal prompt injection attacks against large language models,” 2024.
- [8] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Prompt injection attacks and defenses in llm-integrated applications,” 2023.
- [9] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, “Prompt injection attack against llm-integrated applications,” 2024.
- [10] J. Li, Y. Liu, C. Liu, L. Shi, X. Ren, Y. Zheng, Y. Liu, and Y. Xue, “A cross-language investigation into jailbreak attacks in large language models,” 2024.
- [11] Y. Deng, W. Zhang, S. J. Pan, and L. Bing, “Multilingual jailbreak challenges in large language models,” 2024.
- [12] F. Jiang, Z. Xu, L. Niu, Z. Xiang, B. Ramasubramanian, B. Li, and R. Poovendran, “Artprompt: Ascii art-based jailbreak attacks against aligned llms,” 2024.

- [13] G. Deng, Y. Liu, Y. Li, K. Wang, Y. Zhang, Z. Li, H. Wang, T. Zhang, and Y. Liu, "Masterkey: Automated jailbreaking of large language model chatbots," in *Proceedings 2024 Network and Distributed System Security Symposium*, ser. NDSS 2024. Internet Society, 2024. [Online]. Available: <http://dx.doi.org/10.14722/ndss.2024.24188>
- [14] T. M. Gemma Team, C. Hardin, R. Dadashi, S. Bhupatiraju, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, and et al., "Gemma," 2024. [Online]. Available: <https://www.kaggle.com/m/3301>
- [15] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," 2023.
- [16] S. Gunasekar, Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi et al., "Textbooks are all you need," *arXiv preprint arXiv:2306.11644*, 2023.
- [17] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023.
- [18] B. Wilson, "Introducing Stable LM 2 12B — Stability AI — stability.ai," <https://stability.ai/news/introducing-stable-lm-2-12b>, 2024, [Accessed 16-05-2024].
- [19] Y. Wang, H. Li, X. Han, P. Nakov, and T. Baldwin, "Do-not-answer: A dataset for evaluating safeguards in llms," *arXiv preprint arXiv:2308.13387*, 2023.
- [20] L. Banh and G. Strobel, "Generative artificial intelligence," *Electronic Markets*, vol. 33, no. 1, p. 63, Dec 2023. [Online]. Available: <https://doi.org/10.1007/s12525-023-00680-1>